

Fan Curve Derivations

Year	2026
Department	Aerodynamics
Member(s) Responsible	Dulvin Bandara

Introduction

Currently there is a lack of knowledge regarding derivations of fan curves. This was more brought into light with a requirement of usage of fan curves for CFD purposes to gain a quantitative understanding of how the fans work and more recently providing an insight into an air cooled Accumulator.

Background Theory

Euler Turbomachinery Equation

The problem is setup as a having a system with a rotor, which we have defined as a set of blades attached to some spinning shaft. Flow enters this system at some radius r_1 and exits the system at some radius r_2 . From here we can note that the rotor does work on the fluid which we can note is air for this specific scenario and changes the velocity of said fluid as it passes through the system. The aim we to look at with our fan curve is how much work per unit mass does this rotor do on our fluid.

Angular Momentum

First we start with Newton's Second Law but apply this to rotation. As the rotational equivalent to force is torque, and the rotational equivalent of momentum is angular momentum we can start from there. For some particle with mass m moving at some tangential velocity V_θ at some radius r , we can define the angular momentum around some centre axis as:

$$L = m \cdot r \cdot V_\theta$$

We can note that torque is equal to rate of change of angular momentum with respect to time therefore:

$$\tau = \frac{dL}{dt}$$

Apply this to current system

When deriving the system it is important to set up assumptions to make our derivations easier and more malleable to deal with.

Assumptions	Notes
Steady Flow	Ensures mass flow is constant and system is not time dependent
Homogenous Flow	Fluid properties are uniform throughout the system
Adiabatic Flow	No heat transfer between the fluid and the rotor, all work input is changing the fluids mechanical energy
Incompressible Flow	Fluid's density is constant throughout the whole system
1D Flow	Significantly simplifies the flow, and simplifies a 3D flow field into a 1D field of view
Inviscid Flow	Ideal fluid motion model which neglects effects such as friction, diffusion and viscous effects
Fan is rigid	Geometry is constant
No Inlet Swirl	Any swirling effects are from the outlet of the fan

We can also draw a diagram to help visualise the scenario better:

Starting with torque equation below.

$$\tau = \frac{dL}{dt} = \dot{L}_2 - \dot{L}_1$$

We can apply the definition of angular momentum.

$$\tau = \dot{L}_2 - \dot{L}_1 = \dot{m}_2 r_2 V_{\theta_2} - \dot{m}_1 r_1 V_{\theta_1}$$

From our assumptions we can note that mass flow is kept constant due to the steady flow assumption and also based on the system definitions aswell.

$$\tau = \dot{m}(r_2 V_{\theta_2} - r_1 V_{\theta_1})$$

Get Specific Work

We can note power is the product of torque and angular velocity and power being the rate of change of work over time we can note the following relationship

$$\dot{W} = P = \tau \cdot \omega = \dot{m}\omega(r_2 V_{\theta_2} - r_1 V_{\theta_1})$$

We can simplify this further by noting the relationship between the angular and cartesian velocity

$$U = \omega r$$

$$\dot{W} = \dot{m}(U_2 V_{\theta_2} - U_1 V_{\theta_1})$$

Dividing both sides by the mass flow ultimately provides us with specific work

$$w = U_2 V_{\theta 2} - U_1 V_{\theta 1}$$

This equation is Euler's turbomachinery equation, and provides an important baseline to derive the overall fan curve.

Ideal Fan Curve

Velocity Vector's

Taking a look at the diagram below:

We can note that at any given position on the blade there is one net velocity which can be broken down into components using vector addition

$$\mathbf{V} = \mathbf{U} + \mathbf{W}$$

Where $\mathbf{V}$ represent the net velocity of the fluid and what is being measured. $\mathbf{U}$ represent the velocity of the blade, which is purely tangential and is only dependent on the RPM of the fan and the radius of the fan blade; and finally $\mathbf{W}$ is the relative velocity of the fluid with respect of the blade, which in other terms is the velocity of the fluid in the reference frame of the fan blade.

Decomposing the Vectors

Before decomposing the vector's it is important to set-up proper conventions, primarily with coordinate systems. We will let the axial direction run along parallel to the fan and can be defined as the x direction. We can also look at the tangential direction along the circumference of the fan surface and can be denoted by the θ direction.

The blade velocity $\mathbf{U}$ purely moves tangentially as such breaking it up to axial and tangential components

$$U_a = 0$$

$$U_\theta = U = \omega r$$

The net velocity of the fluid $\mathbf{V}$ moves both axially when pushed through the fan and also moves tangentially when being swirled by the blades inside the fan

$$V_a = \text{Axial Component}$$

$$V_\theta = \text{Tangential Component}$$

Using the previously defined vector triangle:

$$\mathbf{W} = \mathbf{V} - \mathbf{U}$$

and splitting into components we can note the following

$$W_a = V_a - U_a = V_a - 0 = V_a$$

$$W_{\theta} = V_{\theta} - U_{\theta} = V_{\theta} - U$$

It is key to note the following, the axial component of the relative velocity of the fluid from the fan blade is only dependent on the axial component of the absolute velocity; and the tangential component of the relative velocity is the swirl of the fluid subtracted by the speed of the blade. If the fluid has no swirl the blade sees the fluid coming at it with a velocity of magnitude U , as the blade is "moving" toward stationary fluid.

Blade Angle

As the relative velocity $\mathbf{W}$ is what the blade actually sees, it helps to create a direction that be used to define it. As such let β define the angle between $\mathbf{W}$ and the tangential direction (Tangential is chosen due to common/natural convention used).

$$\tan \beta = \frac{W_a}{W_{\theta}}$$

As such following from the vector decomposition above

$$W_{\theta} = \frac{W_a}{\tan \beta} = \frac{V_a}{\cot \beta} = V_a \cot \beta$$

Substitution into Euler's Turbomachinery Equation

Following the above simplification we can rearrange the vector triangle once more, to get an expression for V_{θ} , as that it is what we are interested in to substitute into Euler's.

$$V_{\theta} = W_{\theta} + U = V_a \cot \beta + U$$

However, based on convention definition W_{θ} is in the negative direction due to fan blade sweeping the air backwards, with the relative flow tangentially opposing the motion of the blade. As such

$$V_{\theta} = U - V_a \cot \beta$$

Substituting into Euler's, and taking into the account the assumption of no inlet swirl provides with

$$w = U_2(U_2 - V_a \cot \beta_2) = U_2^2 - U_2 V_a \cot \beta_2$$

Noting the relationship between volumetric flow rate, velocity & surface area

$$V = \frac{Q}{A}$$

This can be substituted into the above work relationship

$$w = U_2^2 - \frac{U_2 Q \cot \beta_2}{A}$$

Finally since a fan curve describes the change in pressure, it can be noted the following relationship between work and pressure

$$\Delta p = \rho w$$

and finally substituted provides

$$\Delta p = \rho w = \rho \left(U_2^2 - \frac{U_2 Q \cot \beta_2}{A} \right)$$

$$\Delta p = \rho U_2^2 - \frac{\rho U_2 Q \cot \beta_2}{A}$$

The above relationship describes the Ideal Fan Curve

Simplified Curve

The above derived ideal curve helps to describe what a perfect fan is able to do theoretically. This is not the case as we can note due to real life losses and other factors

Lack of Blade Exit Angle

The ideal model requires a value for β_2 , the blade exit angle. However, most commonly in datasheets this exit angle is not provided. This poses a problem to try and back-calculate and requires more datapoints not readily available.

Overpredicting Pressure

Due to losses faced in real life, the ideal model tends to overpredict the pressure to what we require. We can overcome this by adding a scaling factor but this can only be added to the shutoff pressure, our y-intercept based on the data given and cannot properly be added to the slope.

Linear Model is Not Correct

The ideal fan curve is linear indicating that ideally pressure drop linearly with the flow. However, this is not the case in real life pressure tend's to fall off more significantly which can be better described with a quadratic relationship.

Quadratic Model

Losses in a viscous fluid model are scaled using the Darcy-Weisbach equation for pressure loss within a pipe

$$\Delta p_{loss} = f D \frac{L \rho V^2}{2}$$

f is our friction factor, $\frac{L}{D}$ is the length to diameter ratio of the pipe, V is the mean flow velocity throughout the pipe & ρ is the fluid density. We can use this model through treating the axial fan as a cylindrical "pipe" which can help to model any pressure losses faced through the fan when exposed to this air. As before noting the relationship between Volumetric Flow Rate and velocity this can be applied to the Darcy-Weisbach equation providing us with

$$\Delta p_{loss} = f \frac{L}{D} \frac{\rho Q^2}{2A^2}$$

and by treating every constant as one singular constant defined below

$$k = f \frac{L}{D} \frac{\rho}{2A^2}$$

$$\Delta p_{loss} = kQ^2$$

We can apply this losses to our ideal fan curve previously defined providing us with the following relationship

$$\Delta p_{real} = \Delta p_{ideal} - \Delta p_{loss}$$

$$\Delta p = \rho U_2^2 - \frac{\rho U_2 \cot \beta_2}{A} Q - kQ^2$$

Where k was previously defined. The model can be simplified from here a bit further

$$a = \rho U_2^2$$

$$b = - \frac{\rho U_2 \cot \beta_2}{A}$$

$$c = -k$$

$$\Delta p = a + bQ + cQ^2$$

As such this curve can be fitted with datapoints given in our fan curve datasheets, and is used in our overall derivations from here.

Code Creation

This short section goes into detail regarding any definitions for the code used to generate the fan curve. Please note for all coding purposes the following applies

```
import numpy as np
```

Blade Tip Speed

Explanation

A very important parameter. The blade tip speed describes the cartesian velocity of the fan at the very tip of the fan blade. This speed describes the maximum velocity that the fan is able to impart onto whatever fluid is being dealt with, for the purpose of this report it is safe to assume that the only fluid being dealt with is air.

Input Variables	Units	Notes
N	RPM	Rotational speed of the fan
r	m	Radius of the fan

The process for calculating the blade tip speed is summarised below.

$$U_2 = \omega \cdot r = \frac{2\pi N}{60} \cdot r$$

Output	Units	Notes
U_2	m/s	Blade Tip Speed

Python Code

```
def BladeTipSpeed(nRPM, dM):
    """
    nRPM : Rotational speed of the fan (RPM)
    dM : Diameter of the fan (m)
    U_2 : Blade tip speed (m/s)
    """

    # Converts RPM to rad/s
    omega = 2*np.pi*nRPM/60
    # Calculates the tip speed converting rotational velocity to cartesian
    velocity
    U_2 = omega*(dM/2)

    return U_2
```

Theoretical Shutoff Pressure

The theoretical shutoff pressure refers to the maximum pressure a rotor spinning at some blade tip speed U_2 can achieve. Basically if the fan was ideal with no losses, perfect blades, etc. this would be the pressure the fan achieves. It is impossible for a physical fan to achieve this but is an upper bound derived through Euler's and more simply is the y-intercept in both the ideal fan curve and our modelled fan curve.

Input Variables	Units	Notes
ρ	kg/m ³	Density of the fluid
U_2	m/s	Blade Tip Speed

The process for calculating is summarised below

$$p_{theoretical} = \rho U_2^2$$

Output	Units	Notes
p_theoretical	Pa	Theoretical Shutoff Pressure

Python Code

```
def TheoShutoffPressure(U_2, rho):
    """
    U_2 : Blade tip speed (m/s)
    rho : Density of the fluid (kg/m^3)
    p_theo : Theoretical shutoff pressure (Pa)
    """

    # Calculates theoretical shutoff pressure
    p_theo = rho*U_2**2

    return p_theo
```

Slip Factor

Slip factor summarises the difference between the theoretical shutoff pressure we calculate vs the shutoff pressure that is measured through datasheets. The main phenomenon that create this "slip" are:

Boundary Layer Growth

Viscous boundary layers which grow on the blade which in turn effectively reduces the surface area of the flow and distort the velocity profile actually used.

Tip Leakage

Vortices created causing induced drag through air leaking from the tips of the blades. The leakage of this high pressure fluid into a low pressure side effectively reduces the velocity profile

Input Variables	Units	Notes
P_real	Pa	Real Shutoff Pressure
P_theoretical	Pa	Theoretical Shutoff Pressure

The process for calculating is summarised below

$$\psi = \frac{\Delta p_{real}}{\Delta p_{theoretical}}$$

Output	Units	Notes
Psi	N/A	Slip Factor

Python Code

```
def SlipFactor(delta_P_actual, delta_P_theoretical):
    """
    delta_P_actual : Actual maximum pressure for fan (Pa)
    delta_P_theoretical : Theoretical maximum pressure for fan (Pa)
    Slip Factor : Ratio of what fan reaches vs what it could theoretically
    achieve
    """

    if delta_P_actual is None:
        return None
    else:
        slipFactor = delta_P_actual/delta_P_theoretical

    return slipFactor
```

Actual Shutoff Pressure

Taking into account the slip factor previously calculated, this can be used to apply a factor onto the theoretical shutoff pressure and find out what is actually being pushed out from the fan

Input Variables	Units	Notes
P_real	Pa	Real Shutoff Pressure
U_2	m/s	Blade Tip Speed

The process is summarised below

$$\Delta p_{actual} = \psi \Delta p_{theoretical}$$

Output	Units	Notes
P_max	Pa	Actual Shutoff Pressure
P_theoretical	Pa	Theoretical Shutoff Pressure
source		Text noting if derived or data missing

Python Code

```
def ActualShutoff(delta_P_actual, U_2, psi_default = 0.45, rho = RHO):
    """
    delta_P_actual : Actual maximum pressure for fan (Pa)
    U_2 : Blade tip speed (m/s)
    """

    delta_P_theoretical = TheoShutoffPressure(U_2, rho)

    if delta_P_actual is not None:
        psi = SlipFactor(delta_P_actual, delta_P_theoretical)
        source = f"Datasheet (psi={psi:.3f})"
    else:
        psi = psi_default
        source = f"Estimated (psi={psi:.2f})"

    delta_P_max = psi * delta_P_theoretical

    return delta_P_max, delta_P_theoretical, psi, source
```

Solving for a,b,c

This helps to solve the constants used for the polynomial for the fan curve.

Input Variables	Units	Notes
P_max	Pa	Actual Shutoff Pressure
Q_max	m ³ /s	Maximum flow rate

The process is summarised below

$$a = \Delta P_{max}$$

$$k = \frac{\Delta p_d - a(1 - \tilde{Q}_{max}^2)}{Q_d(Q_{max} - Q_d)}$$

$$b = \frac{a - kQ_{max}^2}{Q_{max}}$$

Output	Units	Notes
a		Coefficient
b		Coefficient
k		Coefficient

Python Code

```
def SolveABC(delta_P_max, Q_max, Q_d = None, dp_d = None):
    """
    delta_P_max : Theoretical Shutoff Pressure (Pa)
    Q_max : Free delivery flow rate (m^3/s)
    Q_d : Flow rate at design point (m^3/s)
    dp_d : Pressure at design point (Pa)
    """

    a = delta_P_max

    if Q_d is not None and dp_d is not None:
        k = (dp_d - a * (1-Q_d/Q_max))/(Q_d*(Q_max-Q_d))
        b = (a-k*Q_max**2)/Q_max
        method = "3-parameter fit"
    else:
        b = 0
        k = a/Q_max**2
        method = "n=2 approximation"

    return a,b,k,method
```

Fluent Coefficients

The above calculated polynomials for the fan curve and then scaled using the area of the surface of the fan to then substitute into Fluent.

Input Variables	Units	Notes
a		Coefficient
b		Coefficient
k		Coefficient
A_fan	m^2	Area of the surface of the fan

Process summarised below

$$c_0 = a$$

$$c_1 = -b \cdot A_{fan}$$

$$c_2 = -k \cdot A_{fan}^2$$

Output	Units	Notes
c0		Coefficient
c1		Coefficient
c2		Coefficient

Python Code

```

def FluentCoefficients(a, b, k, A_fan):
    """
    a : Coefficient from SolveABC
    b : Coefficient from SolveABC
    k : Coefficient from SolveABC
    A_fan : Area of the fan surface (m^2)
    """

    c0 = a
    c1 = -b * A_fan
    c2 = -k * A_fan**2

    return c0, c1, c2

```

Reconstruct Curve

Honestly just helps plot everything to double check, no need to explain too much

Python Code

```

def ReconstructCurve(a,b,k, Q_max, n_points):
    """
    Helps reconstruct the curve for plotting purposes
    """

    Q = np.linspace(0, Q_max, n_points)
    delta_P = a - b * Q - k * Q ** 2

    delta_P = np.maximum(delta_P,0)

    return Q, delta_P

```